An MLIR-Based Approach to HPC Portability

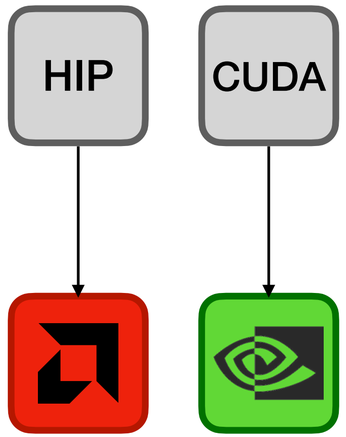
Mojo

Yannick Schürmann, 8 Jul 2026
MPCDF & CAPS TUM, Garching b. München

Motivation

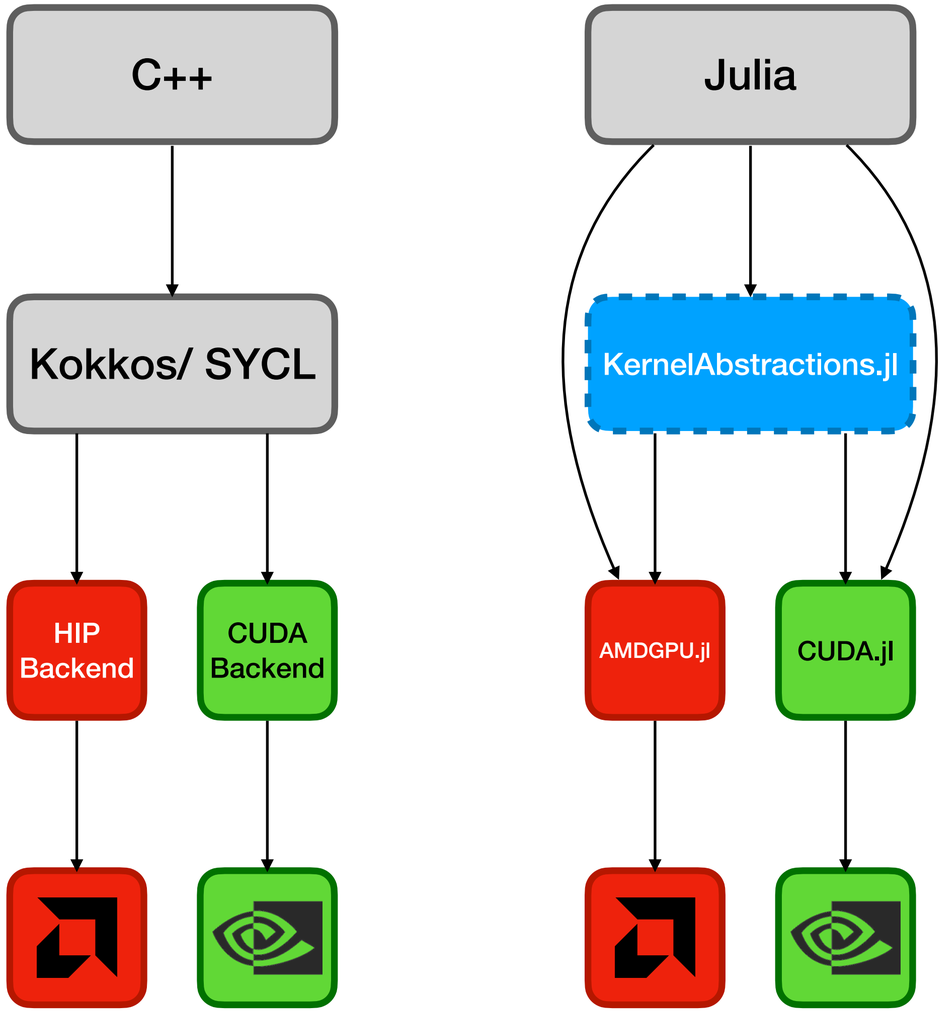
How can we write GPU Kernels?

- Naive approach to writing a NVIDIA/AMD kernel.
  - Just use CUDA [Nickolls et al., 2008] /HIP
- Main issues we try to solve in this presentation:
  - **Vendor lock-in**
  - **Language Split**



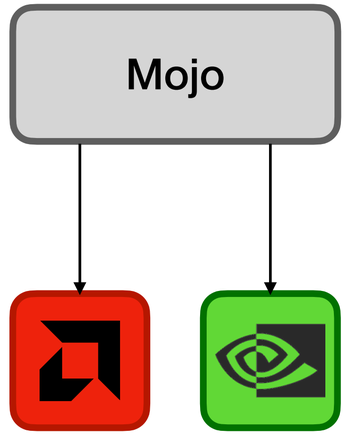
How can we improve on this?

- Addressing Vendor lock in
  - Kokkos [Trott et al.] /SYCL [Keryell et al.], production proven
  - One joined backend for generating both NVIDIA and AMD kernels
- Addressing language split
  - Julia [Bezanson et al.], [Besard et al., 2019], JIT to near-native, mature ecosystem
  - One language to program kernels and prototype



Can we address both issues?

- Mojo attempts addressing both issues [Godoy et al., 2025]
- Superset* of python prototyping and kernel development are both pythonic
  - Python interop runs through CPython at runtime
- Use the compiler to generate both NVIDIA and AMD backends



TOC

- Background
  - LLVM
  - MLIR
- Concept
  - Mojo
- Evalutation
- Conclusion & Outlook

Background

LLVM [Lattner & Adve, 2004]

- One shared IR for all languages
  - One set of optimizations for C, C++, Rust, Swift, …
- Compiler loses significant information
  - Layout, access pattern, other semantics
- To solve issues languages/frameworks often develop own compiler stacks with many mid level IRs

```
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++)
    for (int k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j];
```

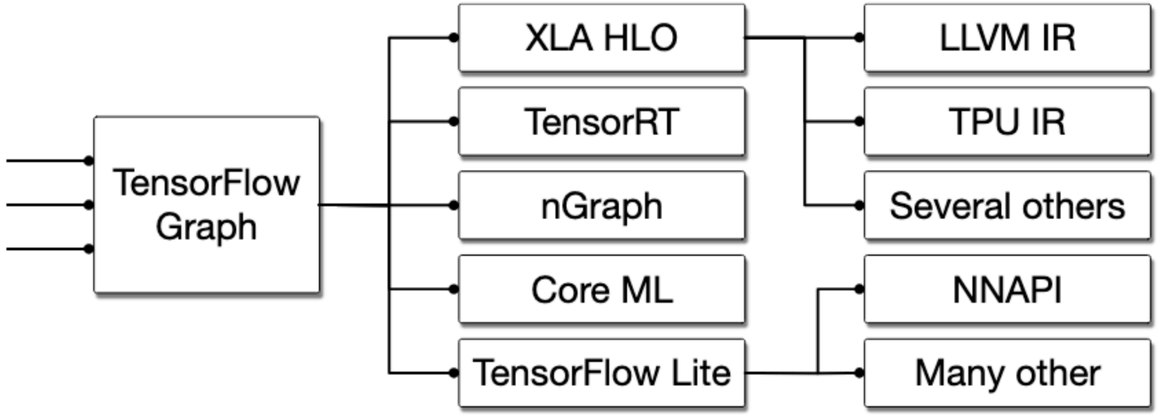
Next slide…

```
%a_p  = getelementptr float,float* %A,i64 %a_i
%a_v  = load float, float* %a_p
%b_p  = getelementptr float,float* %B,i64 %b_i
%b_v  = load float, float* %b_p
%prod = fmul float %a_v, %b_v
%c_p  = getelementptr float,float* %C,i64 %c_i
%c_v  = load float, float* %c_p
%sum  = fadd float %c_v, %prod
store float %sum, float* %c_p
```

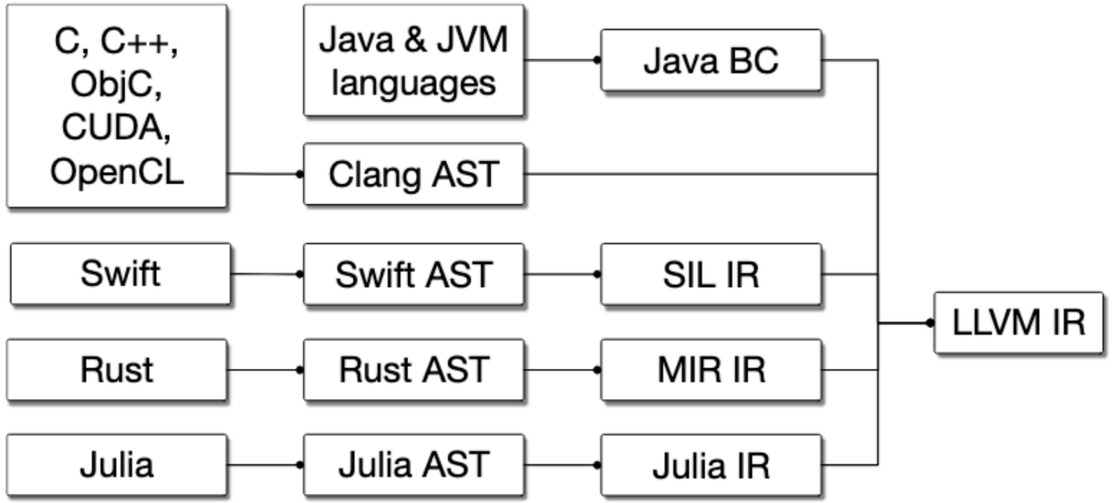
Excerpt

Example of mid level IRs

Frameworks [Lattner et al., 2020]:



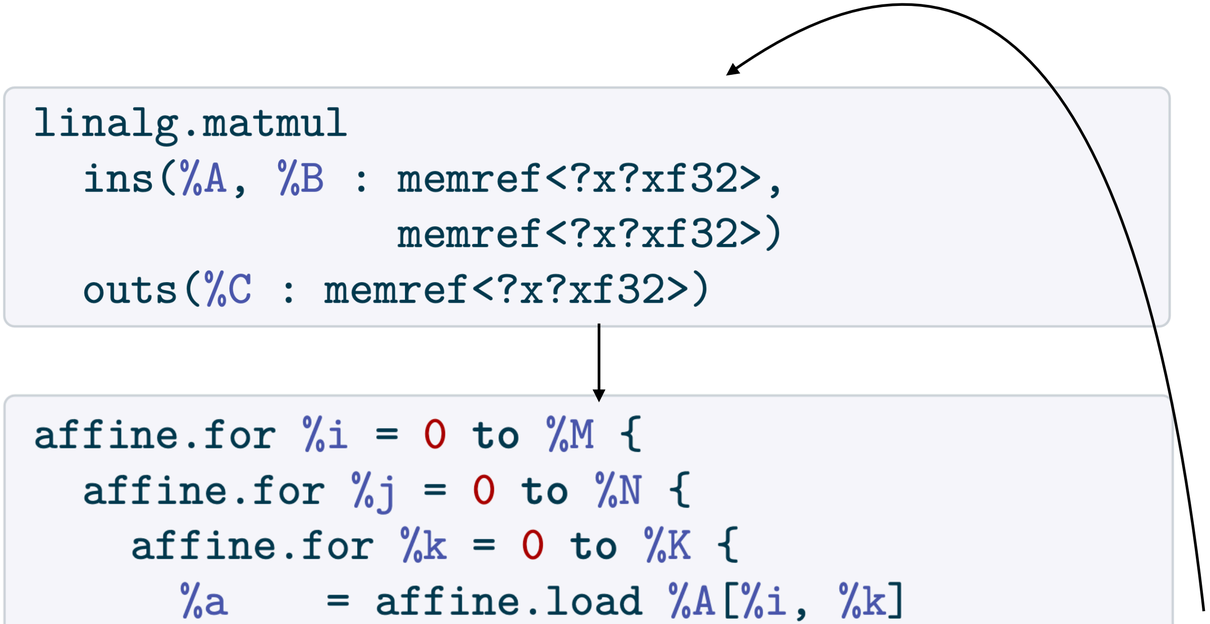
Languages [Lattner et al., 2020]:



MLIR [Lattner et al., 2020]

- Dialects: namespaced sets of ops/types (linalg., affine., nvvm.)
- Progressive lowering, structure kept as long as it's useful
  - linalg -> affine -> scf -> cf -> LLVM IR
- Different parts of one program can sit at different levels at the same time, providing compiler info
- Different IRs are no longer needed, optimizations can be reused within same framework

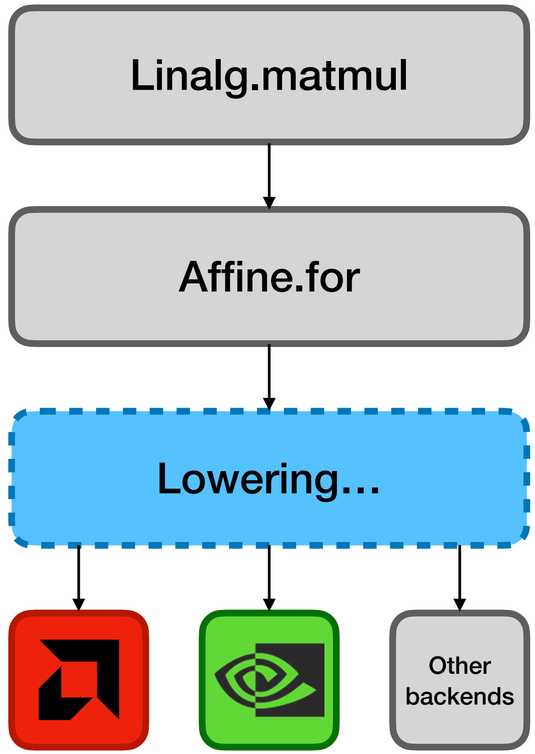
```
linalg.matmul
  ins(%A, %B : memref<?x?xf32>,
               memref<?x?xf32>)
  outs(%C : memref<?x?xf32>)
```



```
affine.for %i = 0 to %M {
  affine.for %j = 0 to %N {
    affine.for %k = 0 to %K {
      %a    = affine.load %A[%i, %k]
                 : memref<?x?xf32>
      %b    = affine.load %B[%k, %j]
                 : memref<?x?xf32>
      %c    = affine.load %C[%i, %j]
                 : memref<?x?xf32>
      %prod = arith.mulf %a, %b : f32
      %sum  = arith.addf %c, %prod : f32
      affine.store %sum, %C[%i, %j]
                 : memref<?x?xf32>
    } } }
```

Payoff

- Portability through dialects
- Same high-level IR + a different set of lowering passes go to a different hardware target
- Hardware-specific code exists only in the final lowering step
  - portability is a property of the compiler infrastructure



Concept

Mojo

- The first language built as MLIR dialects [Godoy et al., 2025]
- Superset of Python, adding exactly what the compiler needs
  - static typing
  - ownership
  - compile-time metaprogramming

Solving our problems

- Two language problem
  - numpy/scipy import directly, no FFI
  - However interop is runtime-only
- Vendor lock-in
  - New hardware target = one new lowering dialect. Portability is in the compiler.

```
def add(x, y):
    return x + y

def add(x: Float32, y: Float32) -> Float32:
    return x + y
```

Code

- parameters are resolved at compile time
- structs and TileTensor have fully static layout, MLIR sees the memory access pattern
- problem shape must be fixed at compile time
  - awkward for adaptive meshes or anything dynamically shaped
- Same def, same language*

```
...

comptime dtype     = DType.float32
comptime N         = 1024
comptime tile      = 16
comptime layout    = row_major[N * N]()
comptime num_tiles = ceildiv(N, tile)


def matmul_kernel(
    C: TileTensor[dtype, type_of(layout),
                  MutAnyOrigin],
    A: TileTensor[dtype, type_of(layout),
                  MutAnyOrigin],
    B: TileTensor[dtype, type_of(layout),
                  MutAnyOrigin],
):
    var row = block_idx.y * block_dim.y
            + thread_idx.y
    var col = block_idx.x * block_dim.x
            + thread_idx.x
    if row < N and col < N:
        var acc = Scalar[dtype](0)
        for k in range(N):
            acc += A[row * N + k]
                 * B[k * N + col]
        C[row * N + col] = acc
```

Evaluation

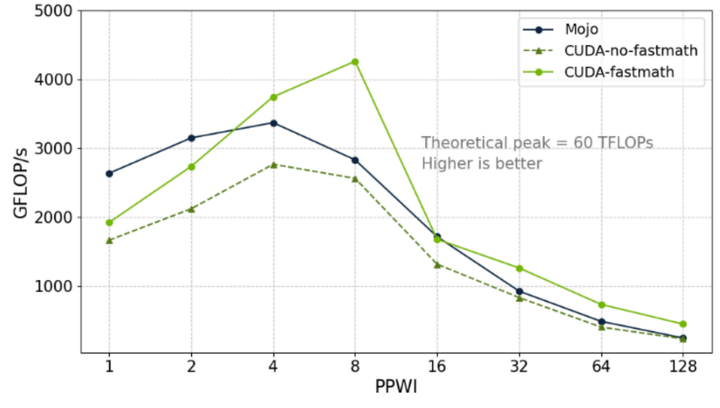
Setup

- Benchmarks from [Godoy et al., SC-W 2025] we survey their results
- Four HPC proxy kernels, Mojo vs. vendor baselines
  - CUDA on NVIDIA H100 and HIP on AMD MI300A
- memory-bound (bottleneck: moving data)
  - seven-point stencil, BabelStream
- compute-bound (bottleneck: arithmetic)
  - miniBUDE, Hartree-Fock
- Evaluate with Φ metric (T is set of GPU platforms)

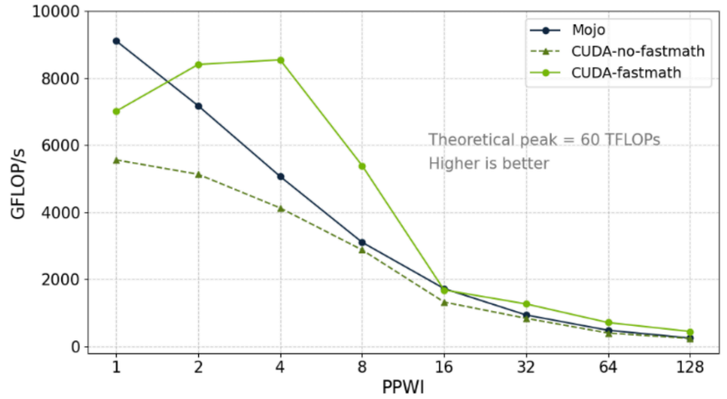
$$\Phi = \frac{\sum_{i \in T} e_i}{|T|}, \quad e_i = \frac{\text{Mojo}_{\text{perf},i}}{\text{vendor}_{\text{perf},i}}$$

Results [Godoy et al., SC-W 2025]

- Memory bound
  - Seven-point stencil, Φ = 0.92
  - BabelStream, Φ = 0.96
- Compute bound
  - miniBUDE Φ = 0.54
  - Hartree-Fock Φ = 0.92 (!)
- Φ collapses two platforms into one number. miniBUDE: Φ = (0.82 + 0.59 + 0.38 + 0.38) / 4 = 0.54 (!)



(a) wg8



(b) wg64

Kernel execution duration (ms)	NVIDIA H100 Mojo	NVIDIA H100 CUDA	AMD MI300A Mojo	AMD MI300A HIP
a=1024 ngauss=6	147,250	2,652	–	846
a=256 ngauss=3	187	472	25,266	178
a=128 ngauss=3	21	53	2,765	23
a=64 ngauss=3	3	7	436	4

Verdict [Godoy et al., SC-W 2025]

- memory-bound
  - near vendor parity on both platforms
- compute-bound
  - real shortfalls, missing fast-math, atomic-operation issues on AMD

Mojo Efficiency	NVIDIA H100	AMD MI300A
7-point stencil		
FP32	0.82	1.00
FP64	0.87	1.00
$\overline{\Phi} = 0.92$		
BabelStream		
Copy	1.01	1.00
Mul	1.02	1.00
Add	1.01	1.00
Triad	1.01	1.00
Dot	0.78	1.00
$\overline{\Phi} = 0.96$		
miniBUDE		
PPWI=8 wg=8	0.82	0.38
PPWI=4 wg=64	0.59	0.38
$\overline{\Phi} = 0.54$		
Hartree–Fock		
a=1024 ngauss=6	17E-3	–
a=256 ngauss=3	2.52	7E-3
a=128 ngauss=3	2.52	8E-3
a=64 ngauss=3	2.33	8E-3
$\overline{\Phi} = 0.92$		

Conclusions and outlook

Critique

- Parses like Python, writing means thinking like a systems programmer
  - explicit thread indexing, compile-time sizes, manual memory layouts
- Python's ergonomics are absent
- Interop is runtime-only. Mojo world and Python stay separate (Invisible costs GIL and Conversion…)
- Lock-in irony. Mojo not yet open source, Modular is the new vendor
- Compilation times can get heavy

```
...

comptime dtype     = DType.float32
comptime N         = 1024
comptime tile      = 16
comptime layout    = row_major[N * N]()
comptime num_tiles = ceildiv(N, tile)


def matmul_kernel(
    C: TileTensor[dtype, type_of(layout),
                  MutAnyOrigin],
    A: TileTensor[dtype, type_of(layout),
                  MutAnyOrigin],
    B: TileTensor[dtype, type_of(layout),
                  MutAnyOrigin],
):
    var row = block_idx.y * block_dim.y
            + thread_idx.y
    var col = block_idx.x * block_dim.x
            + thread_idx.x
    if row < N and col < N:
        var acc = Scalar[dtype](0)
        for k in range(N):
            acc += A[row * N + k]
                 * B[k * N + col]
        C[row * N + col] = acc
```

Conclusions & Outlook

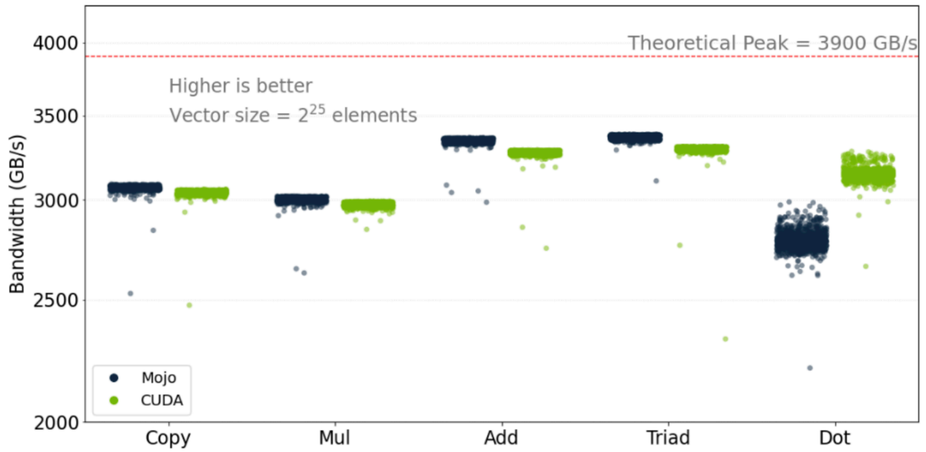
- Today, Mojo solves neither problem fully
- Gaps are implementation holes, not design flaws
  - pre-1.0
- Architecture is unique
- Fall 2026: Modular commits to open-sourcing

Summary

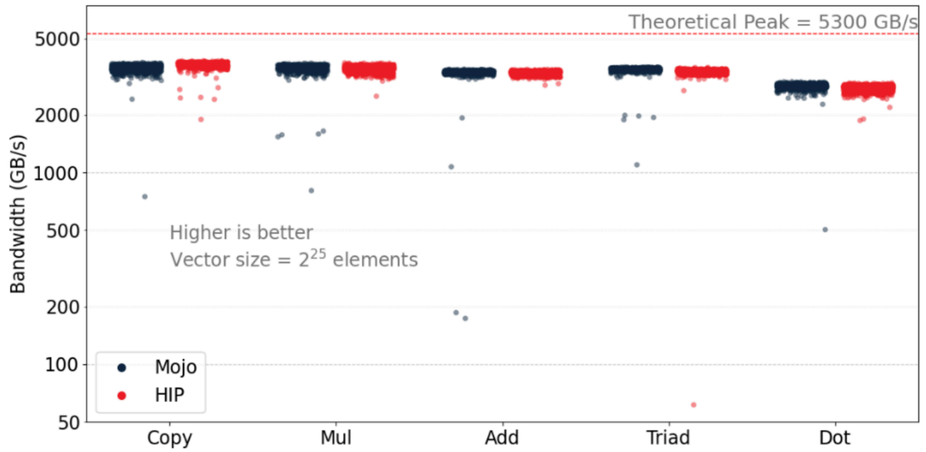
- Problems: two-language split and vendor lock-in
- MLIR, multi-level IR with dialects and progressive lowering. Hardware-specific code only in the last step. Portability becomes a compiler property
- Mojo’s solution, a language built as MLIR dialects
- Near vendor parity memory-bound (Φ ~ 0.9), real gaps compute-bound
- Verdict, right architecture, unfinished engineering

Extra: Babelstream and seven-point results

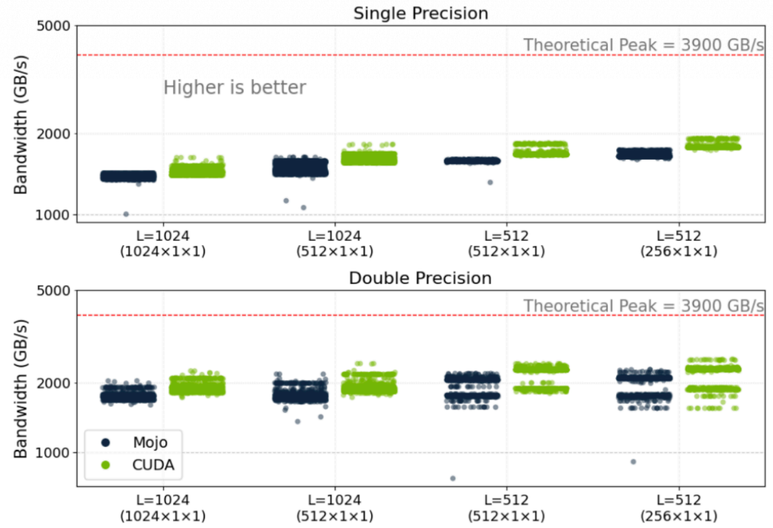
[Godoy et al., SC-W 2025]



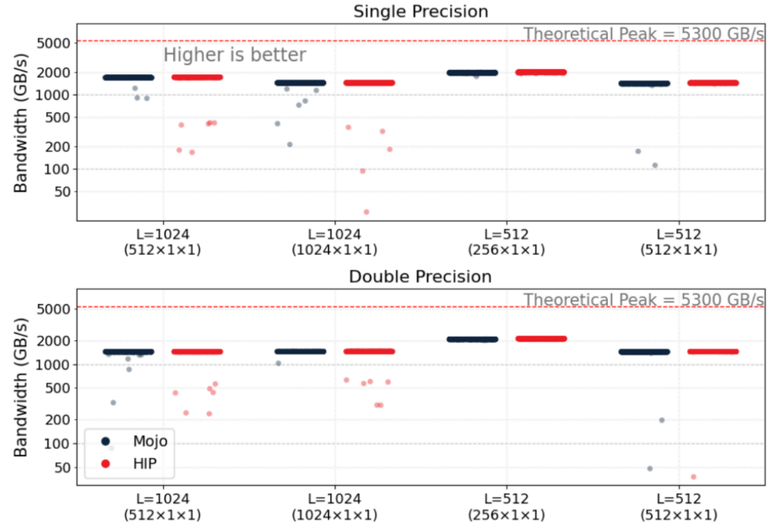
(a) H100



(b) MI300A



(a) H100



(b) MI300A